



UNIVERSITÀ DEGLI STUDI DI PALERMO
DIPARTIMENTO DI INGEGNERIA — A.A. 2025/2026

Object Design Document



Gruppo di lavoro

Simone Sottile
Matteo Duca
Giuseppe Giacomarra
Gilberto Mogavero

Indice

1	Introduzione	4
1.1	Object design trade-offs	4
1.2	Linee guida per la documentazione delle interfacce	4
1.3	Linee guida per la documentazione dei DBMS	5
1.4	Definizioni preliminari	5
2	Packages	6
2.1	it.afam	6
2.1.1	entity	6
2.1.2	utility	7
2.1.3	gestioneAutenticazione	7
2.1.4	gestioneStudiante	8
2.1.5	gestioneCondivisione.	8
2.1.6	consultazione	8
2.1.7	gestioneOperazioniAutomatiche	8
2.2	org.xerial.sqlite-jdbc.	8
2.3	javafx	9
2.4	java.	9
2.5	jakarta.mail.	10
3	Object Design UML	11
3.1	Entity	11
3.2	Utility	12
3.3	GestioneAutenticazione.interface	13

3.4	GestioneAutenticazione.control	14
3.5	GestioneStudente.interface	15
3.6	GestioneStudente.control	16
3.7	GestioneCondivisione.interface	17
3.8	GestioneCondivisione.control	18
3.9	Consultazione.interface	19
3.10	Consultazione.control	20
3.11	GestioneOperazioniAutomatiche.interface	21
3.12	GestioneOperazioniAutomatiche.control	22

1 Introduzione

1.1 Object design trade-offs

Per la realizzazione del Sistema si è scelto di utilizzare un approccio modulare per favorire la scalabilità, la facilità di implementazione e la gestione di eventuali problemi.

In particolare, è stata adottata un'architettura di tipo Repository, che garantisce il disaccoppiamento dei sotto-sistemi, i quali comunicano esclusivamente con il sotto-sistema di Storage (DatabaseManager), senza interagire direttamente tra loro.

A loro volta i sotto-sistemi sono composti da un package di interfacce grafiche FXML (boundary) che comunica solo con il corrispondente package di controllo (control), il quale contiene la logica applicativa e gestisce le richieste verso lo strato di Storage. I package boundary e control di ciascun sotto-sistema risiedono sullo stesso nodo, mentre il DatabaseManager costituisce un nodo logico separato.

Il Sistema è stato suddiviso in cinque sotto-sistemi sulla base del criterio di coesione funzionale: gestioneAutenticazione, gestioneStudente, gestioneCondivisione, consultazione e gestioneOperazioniAutomatiche. Come DBMS si è scelto SQLite: trattandosi di un'applicazione desktop locale, il database embedded garantisce la massima semplicità di distribuzione e zero dipendenze di rete, rimanendo pienamente compatibile con le API standard JDBC.

1.2 Linee guida per la documentazione delle interfacce

Per quanto riguarda le interfacce grafiche si è scelto di usare il pacchetto di librerie JavaFX, che permette di realizzare le interfacce grafiche attraverso documenti di marcatura FXML.

JavaFX è un pacchetto di librerie component-based; ciò permette di avere una grande riusabilità del codice, oltre a una maggiore leggibilità in fase di scrittura.

Più in particolare, per la realizzazione delle interfacce grafiche del Sistema è stato utilizzato JavaFX Scene Builder, un tool per la creazione di documenti di marcatura FXML che sfrutta un sistema drag-and-drop.

Ogni schermata del sistema è composta da un file FXML e dal corrispondente controller Java. La navigazione tra le schermate è gestita in modo centralizzato dalla classe SceneManager (Singleton), che mantiene un unico Stage e sostituisce la Scene corrente ad ogni transizione, garantendo coerenza con i flussi di navigazione descritti nel RAD.

1.3 Linee guida per la documentazione dei DBMS

Per quanto riguarda il DBMS, si è scelto di usare SQLite, un sistema open source di gestione di database relazionali embedded, che non richiede l'installazione di un server separato e archivia l'intero database in un singolo file locale (afam.db).

Per la comunicazione tra il Sistema e il DBMS si è scelto di usare JDBC (Java DataBase Connectivity) tramite il driver org.xerial:sqlite-jdbc. JDBC è un connettore standard che consente l'accesso e la gestione della persistenza dei dati da qualsiasi programma Java, indipendentemente dal tipo di DBMS utilizzato.

Tutte le operazioni CRUD verso il database sono centralizzate nella classe DatabaseManager (Singleton), che espone metodi DAO specifici per ogni entità del sistema. In caso di SQLException, il sistema intercetta l'errore e mostra all'utente la schermata MessaggioDiAvviso con le opzioni Riprova e Annulla, come previsto dal caso d'uso Caduta di connessione (UC 0.1) del RAD.

1.4 Definizioni preliminari

- **Classe di modello (Entity):** Contiene i dati che devono essere gestiti e persistiti nel sistema. Implementa solo attributi privati con relativi getter e setter, senza logica di business.
- **SceneManager:** Singleton che incapsula lo Stage JavaFX dell'applicazione. Ogni controller boundary lo invoca per effettuare la transizione alla schermata successiva tramite il metodo switchTo(String fxmlName), garantendo che esista un unico Stage attivo.
- **SessioneCorrente:** Singleton che mantiene il riferimento all'oggetto Studente autenticato durante la sessione attiva. Viene popolato al completamento del login e azzerato al logout o al raggiungimento del timeout di inattività (30 minuti), come specificato nei Requisiti Organizzativi del RAD.
- **DatabaseManager:** Singleton che gestisce la connessione JDBC a SQLite e fornisce tutti i metodi DAO (Data Access Object) per le operazioni di lettura, inserimento, aggiornamento ed eliminazione su ciascuna entità del sistema.
- **Utils:** Classe di utilità con metodi statici condivisi da tutti i sotto-sistemi per la validazione degli input (email, password, nome, codice fiscale, telefono), l'hashing delle password (SHA-256), e la generazione di token univoci, codici di sblocco e OTP.
- **MessaggioDiAvviso:** Finestra di dialogo modale, implementata localmente in ciascun sotto-sistema che la utilizza, per la visualizzazione di messaggi di avviso (incl. la caduta di connessione con il database, UC 0.1), con pulsanti Riprova e Annulla. Corrisponde alla classe boundary omonima del Modello degli Oggetti.

- **MessaggioDiErrore:** Finestra di dialogo modale, implementata localmente in ciascun sotto-sistema che la utilizza, per la visualizzazione di un messaggio di errore conseguente a un'azione effettuata dallo studente. Contiene il pulsante OK.
- **MessaggioDiConferma:** Finestra di dialogo modale, implementata localmente in ciascun sotto-sistema che la utilizza, per informare lo studente del successo di un'operazione (pulsante OK) o per richiedere una conferma prima di un'azione (pulsanti Conferma e Annulla).

2 Packages

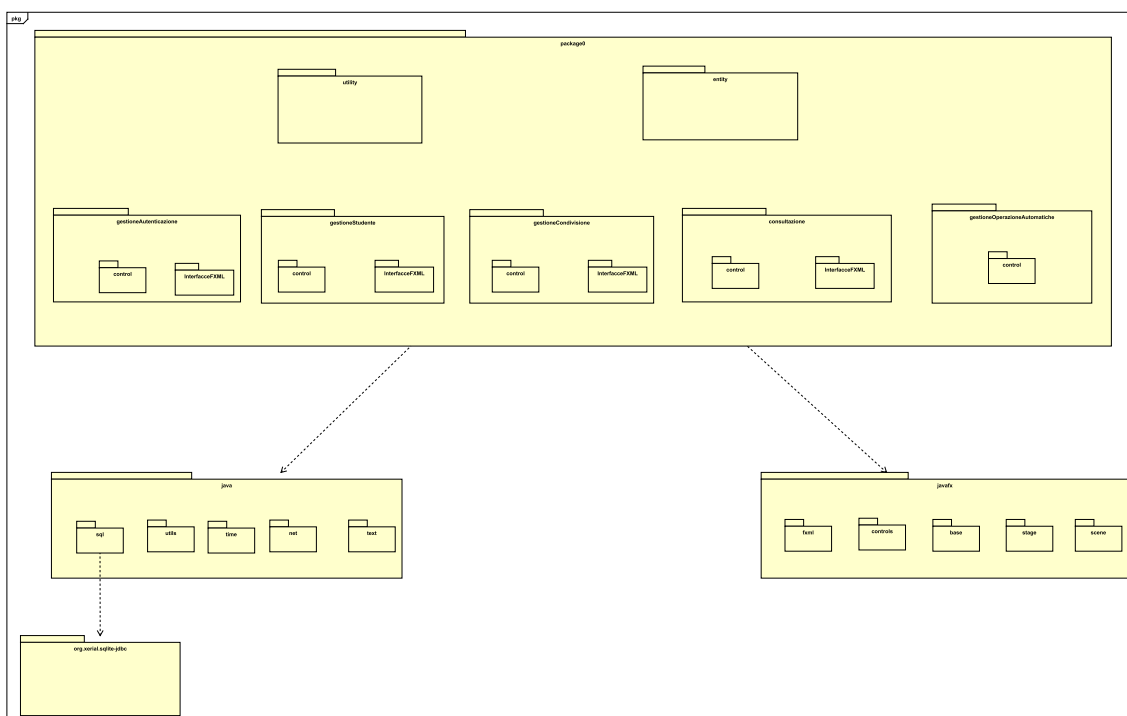


Figura 1: Diagramma UML dei package

2.1 it.afam

Si tratta del package principale dell'applicazione. Al suo interno sono presenti i seguenti package:

2.1.1 entity

Contiene la classe che modella l'entità di cui è necessaria la rappresentazione nel sistema.

- **EntityStudente:** Classe di modello che rappresenta lo studente autenticato sulla piattaforma. Contiene i dati anagrafici e di accesso dell'utente (nome, cognome, codice fiscale, email, password hashata, telefono, percorso della foto). È l'unica classe entity del sistema, in quanto è l'unico oggetto persistito che viene caricato in memoria durante la sessione attiva.

2.1.2 utility

Contiene le classi comuni a tutti i cinque sotto-sistemi. In particolare:

- **SceneManager:** Singleton che gestisce la navigazione tra le schermate dell'applicazione. Mantiene un riferimento unico allo Stage JavaFX e consente a qualsiasi controller di cambiare schermata tramite il metodo `switchTo(String fxmlName)`, senza che i sotto-sistemi debbano conoscersi a vicenda.
- **SessioneCorrente:** Singleton che mantiene in memoria il riferimento all'oggetto `EntityStudente` autenticato durante la sessione corrente. Viene popolato al completamento del login e azzerato al logout o allo scadere del timeout di inattività di 30 minuti, come previsto dai Requisiti Organizzativi del RAD.
- **DatabaseManager:** Singleton che gestisce la connessione JDBC al database SQLite (`afam.db`) e funge da unico punto di accesso allo strato di persistenza per tutti i sotto-sistemi. Espone metodi DAO per ciascuna tabella del database. Implementa la logica di gestione della caduta di connessione prevista dal caso d'uso UC 0.1.
- **Utils:** Classe con metodi statici condivisi da tutti i sotto-sistemi. Fornisce funzionalità di validazione degli input (email, password, nome, cognome, codice fiscale, telefono), hashing delle password tramite SHA-256, e generazione di stringhe univoche per token di link, codici di sblocco e codici OTP.
- **BoundaryDBMS:** Rappresenta l'interfaccia di comunicazione tra il Sistema e il DBMS. Viene invocata dal `DatabaseManager` per inoltrare le richieste di lettura e scrittura dei dati persistenti, gestendo il recupero, la memorizzazione e l'aggiornamento delle informazioni relative a studenti, file, contenuti, link e codici.

2.1.3 gestioneAutenticazione

Contiene tutti i package relativi alle funzionalità di autenticazione e accesso al sistema da parte dello Studente. Si suddivide in `interfacceFXML`, che contiene i file FXML e i relativi controller delle interfacce utente, e `control`, che contiene le classi che gestiscono la logica dei corrispondenti casi d'uso. Nel complesso permette allo studente di effettuare il login tramite credenziali locali con verifica OTP a due fattori, il login federato tramite SPID/eIDAS, la registrazione di un nuovo account con attivazione via codice, il recupero della password e il logout.

2.1.4 gestioneStudente

Contiene tutti i package relativi alla gestione del profilo personale e dei contenuti artistici dello Studente. Si suddivide in interfacceFXML e control. Nel complesso permette allo studente di modificare i propri dati personali, gestire il background artistico, organizzare i contenuti in categorie artistiche, caricare, modificare ed eliminare file multimediali e documentali, e regolare la privacy di ciascun contenuto.

2.1.5 gestioneCondivisione

Contiene tutti i package relativi alla gestione della condivisione dei contenuti tramite link temporizzati e tramite codici di sblocco interni. Si suddivide in interfacceFXML e control. Nel complesso permette allo studente di selezionare contenuti, generare link univoci temporizzati o codici alfanumerici univoci, visualizzarne l'elenco e revocarne la validità.

2.1.6 consultazione

Contiene tutti i package relativi alla consultazione dei profili e dei contenuti degli studenti da parte di qualsiasi utente della piattaforma (Studente, Utente, Utente Esterno). Si suddivide in interfacceFXML e control. Nel complesso permette di cercare uno studente, visualizzarne i contenuti pubblici, accedere ai contenuti privati tramite codice di sblocco, e fruire dei contenuti sia direttamente che tramite link di condivisione.

2.1.7 gestioneOperazioniAutomatiche

Contiene le classi relative all'esecuzione di funzionalità automatiche attivate dall'attore Tempo. Non è presente alcun package di interfacce grafiche in quanto questo sottosistema non interagisce direttamente con l'utente. L'unica classe boundary presente è BoundaryTempo, che rappresenta l'interfaccia verso l'attore Tempo.

2.2 org.xerial.sqlite-jdbc

Contiene il driver JDBC per la comunicazione con il DBMS SQLite utilizzato per la realizzazione del sistema. Il driver è distribuito come dipendenza Maven e viene utilizzato dalla classe DatabaseManager per stabilire la connessione al file di database afam.db e per eseguire le query SQL tramite le interfacce standard java.sql.

2.3 javafx

JavaFX è una tecnologia che consente di sviluppare GUI, grafica e componenti multimediali. Il package è utilizzato per la realizzazione e la gestione delle interfacce grafiche utente.

- **fxml:** Permette di caricare i file FXML e gestisce il binding tra le variabili della classe controller associata e le rispettive componenti grafiche tramite id ed EventHandler.
- **controls:** Definisce i controlli UI disponibili per il toolkit JavaFX, quali TextField, Button, ListView, ComboBox e TableView, utilizzati nelle schermate del sistema.
- **base:** Utilizzato per creare oggetti di collezione osservabili (ObservableList) utili alla creazione di interfacce utente dinamiche. Gli elementi di queste collezioni generano notifiche quando vengono modificati, aggiornando automaticamente la vista associata.
- **stage:** Fornisce le classi contenitore di primo livello (Stage, Scene) per il contenuto JavaFX. Lo Stage è gestito in modo centralizzato dalla classe SceneManager.
- **scene:** Utilizzata per la creazione e la gestione delle scene grafiche. Permette di gestire l'interazione e la visualizzazione dei contenuti all'interno dell'applicazione.
- **media:** Fornisce le classi Media, MediaPlayer e MediaView per la riproduzione di contenuti audio e video in streaming controllato all'interno della SchermataRiproduttoreMultimediale.
- **web:** Fornisce la classe WebView utilizzata dalla SchermataVisualizzatoreDocumenti per la visualizzazione inline di documenti PDF tramite URL locale (file://), inibendo nativamente qualsiasi funzione di download.

2.4 java

Contiene i package e le librerie standard di Java utilizzati nella realizzazione del sistema.

- **sql:** Utilizzato per gestire le comunicazioni con il DBMS. Fornisce Connection, PreparedStatement e ResultSet per la creazione di connessioni e l'esecuzione di query parametrizzate tramite DatabaseManager.
- **util:** Contiene classi e interfacce per la gestione di collezioni (List, Map), la generazione di valori casuali (UUID, Random) e la gestione del timer di sessione (Timer, TimerTask).
- **time:** API per le date e il tempo. Utilizzato per la gestione delle date di creazione e scadenza dei link di condivisione (LocalDate) e per il calcolo del periodo di validità di 14 giorni.

- **io:** Fornisce le classi per la gestione dei file fisici (File, Files, Path). Utilizzato dalla classe UploadFileControl per la copia dei file caricati dallo studente nella cartella locale afam_files/.
- **security:** Fornisce la classe MessageDigest utilizzata dalla classe Utils per l'hashing delle password tramite l'algoritmo SHA-256 prima del salvataggio sul database.

2.5 jakarta.mail

Contiene la libreria Jakarta Mail (Eclipse Angus), utilizzata per l'invio delle e-mail contenenti i codici OTP, di attivazione e di recupero password tramite protocollo SMTP. È distribuita come dipendenza Maven e viene invocata direttamente dalle classi LoginControl, RegistrazioneControl e RecuperaPasswordControl del sotto-sistema gestioneAutenticazione, senza passare per una classe boundary dedicata, in quanto il gateway SMTP è considerato esterno al sistema.

3 Object Design UML

In questa sezione sono inseriti i diagrammi UML esportati in formato PDF vettoriale a partire dagli SVG presenti nella cartella pdf/.

3.1 Entity

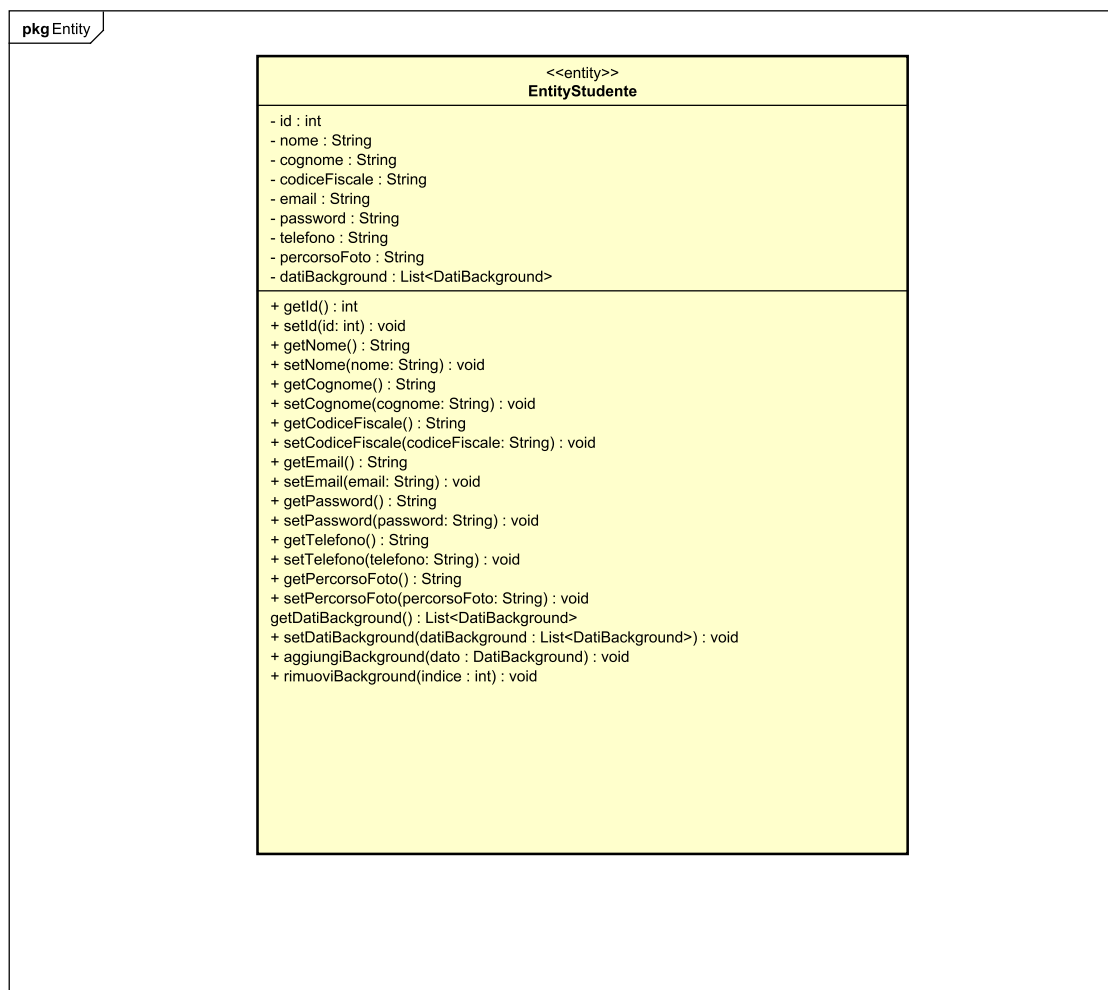


Figura 2: Diagramma UML del package Entity

3.2 Utility

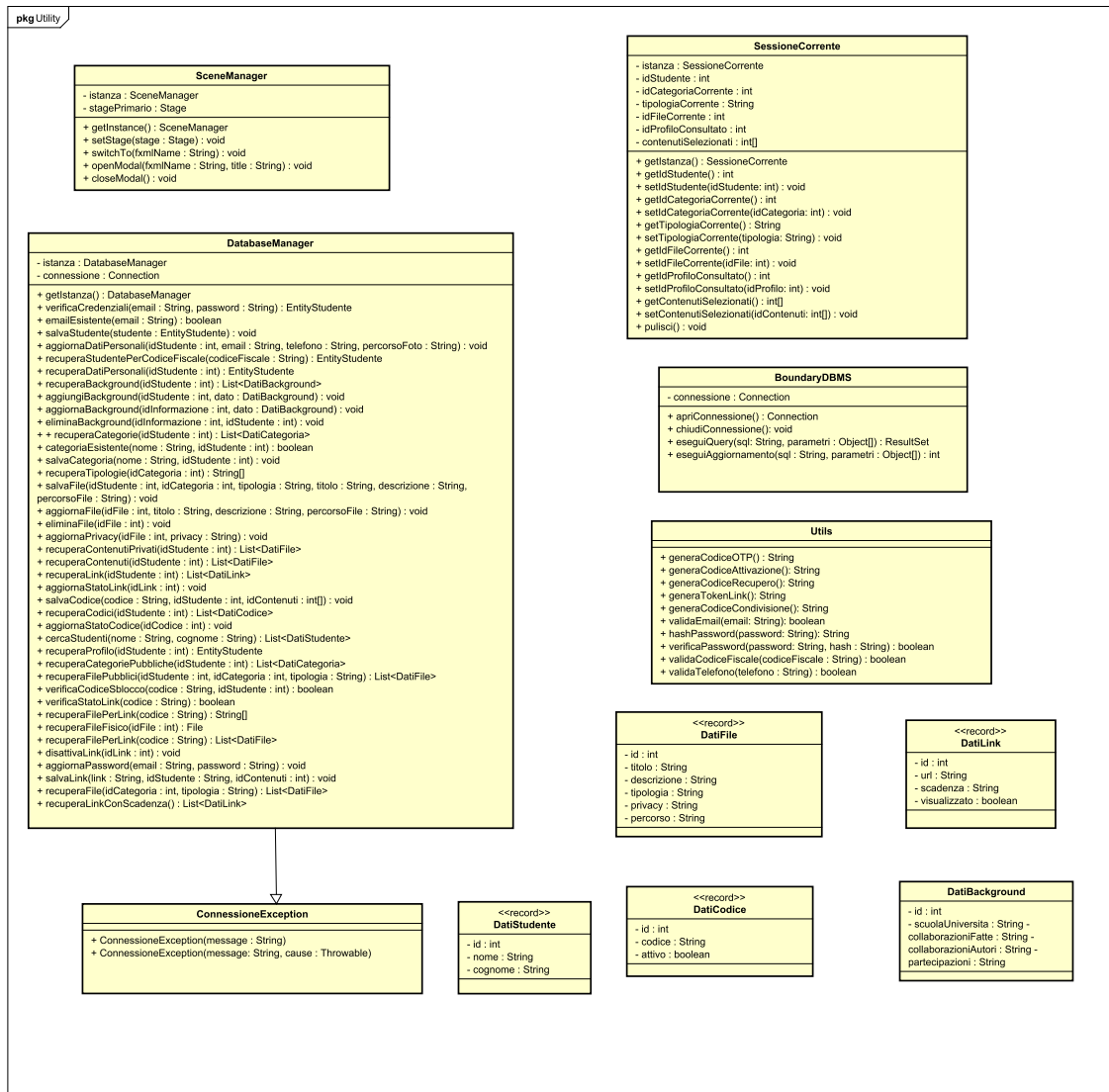


Figura 3: Diagramma UML del package Utility

3.3 GestioneAutenticazione.interface

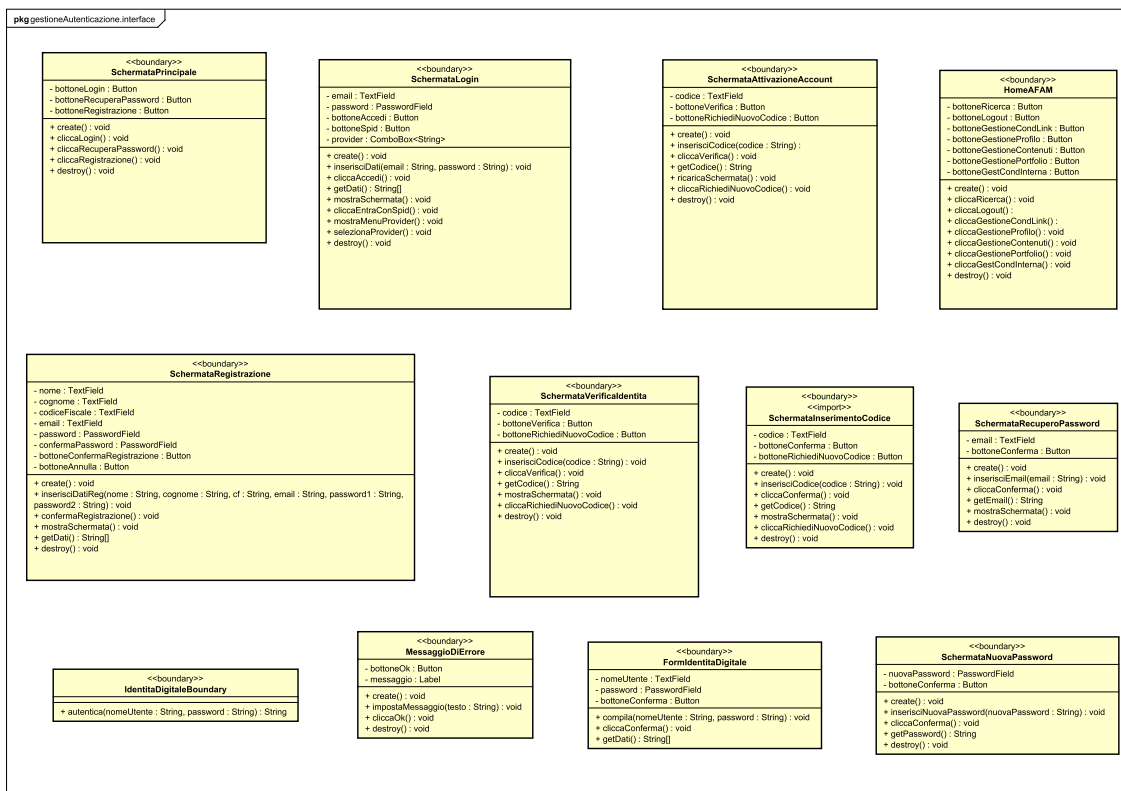


Figura 4: Diagramma UML del package GestioneAutenticazione.interface

3.4 GestioneAutenticazione.control

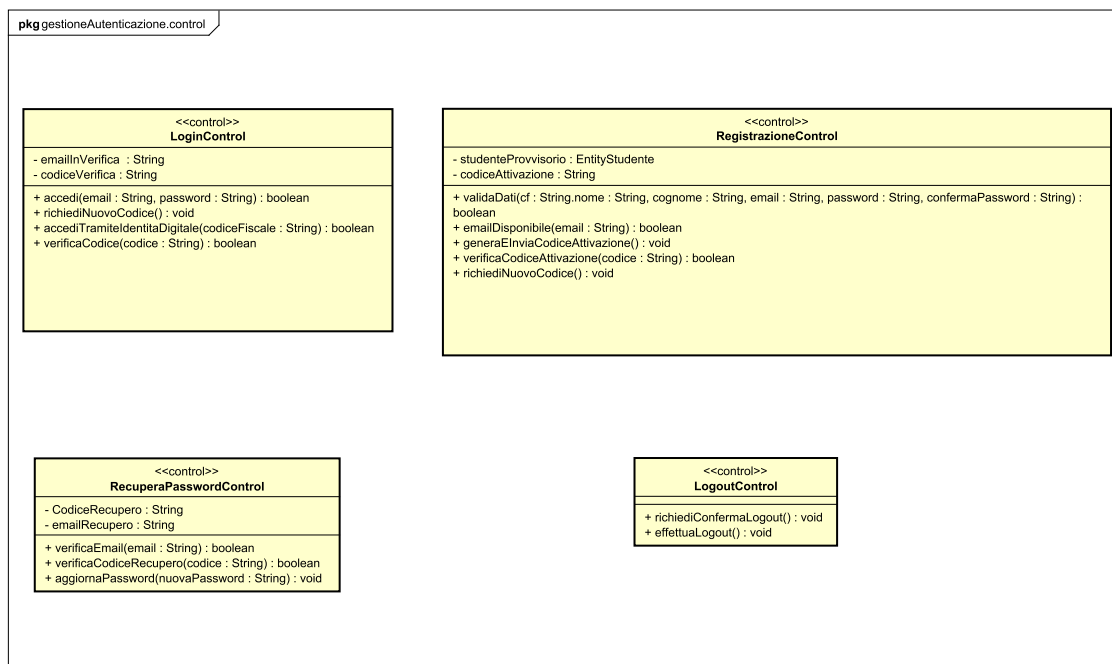


Figura 5: Diagramma UML del package GestioneAutenticazione.control

3.5 GestioneStudiante.interface

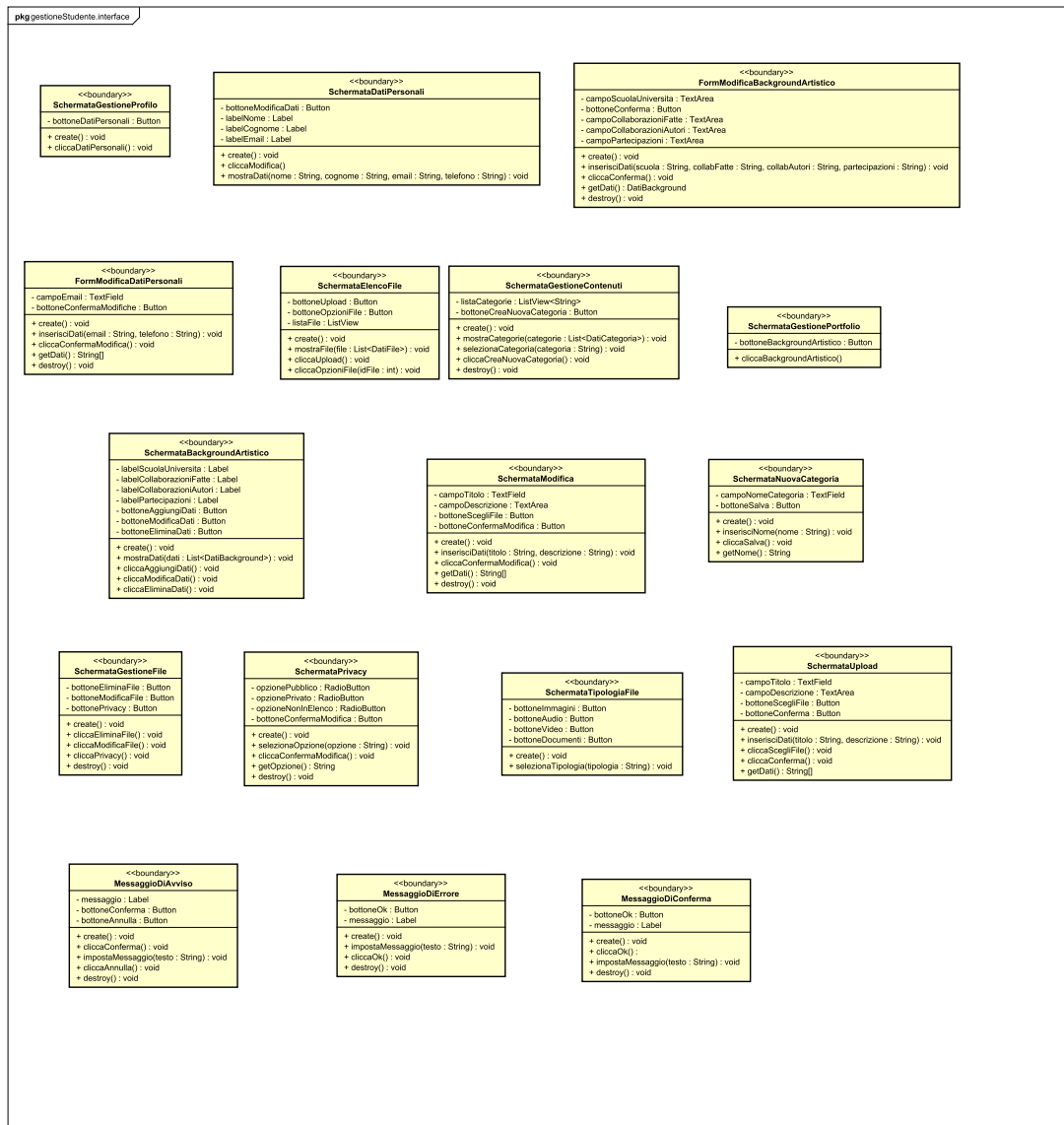


Figura 6: Diagramma UML del package GestioneStudiante.interface

3.6 GestioneStudiante.control

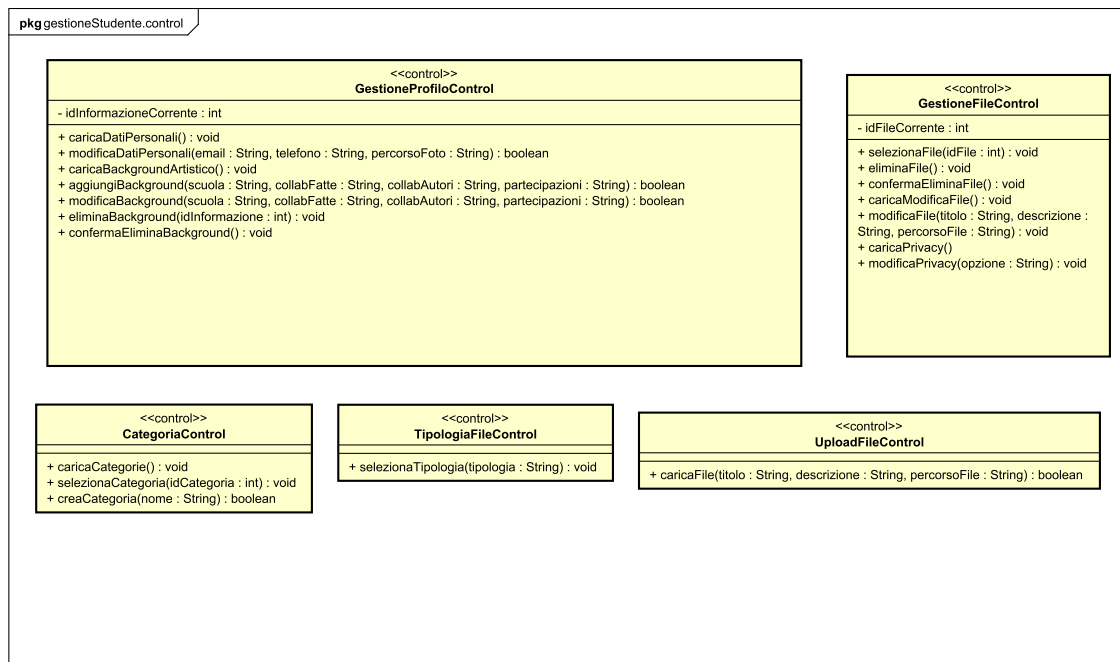


Figura 7: Diagramma UML del package GestioneStudiante.control

3.7 GestioneCondivisione.interface

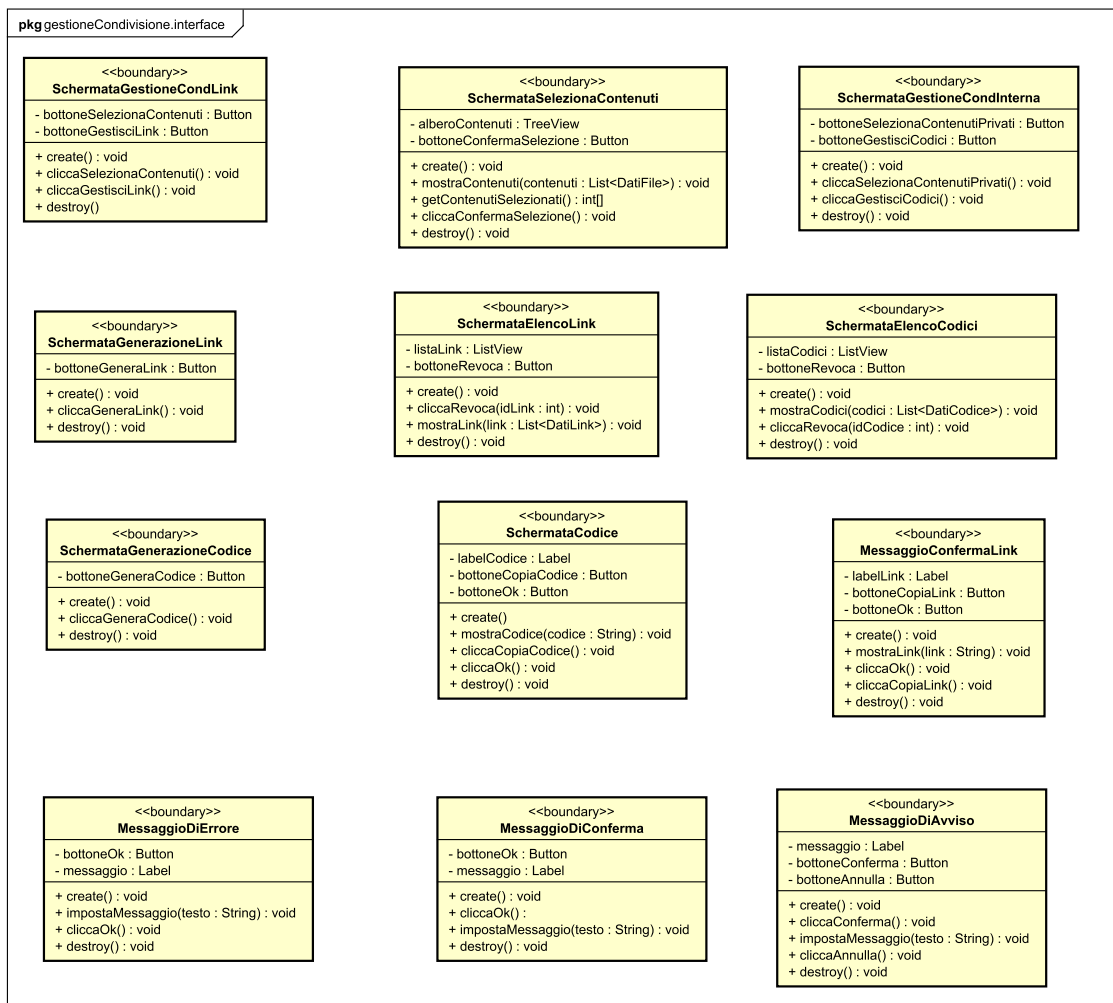


Figura 8: Diagramma UML del package GestioneCondivisione.interface

3.8 GestioneCondivisione.control

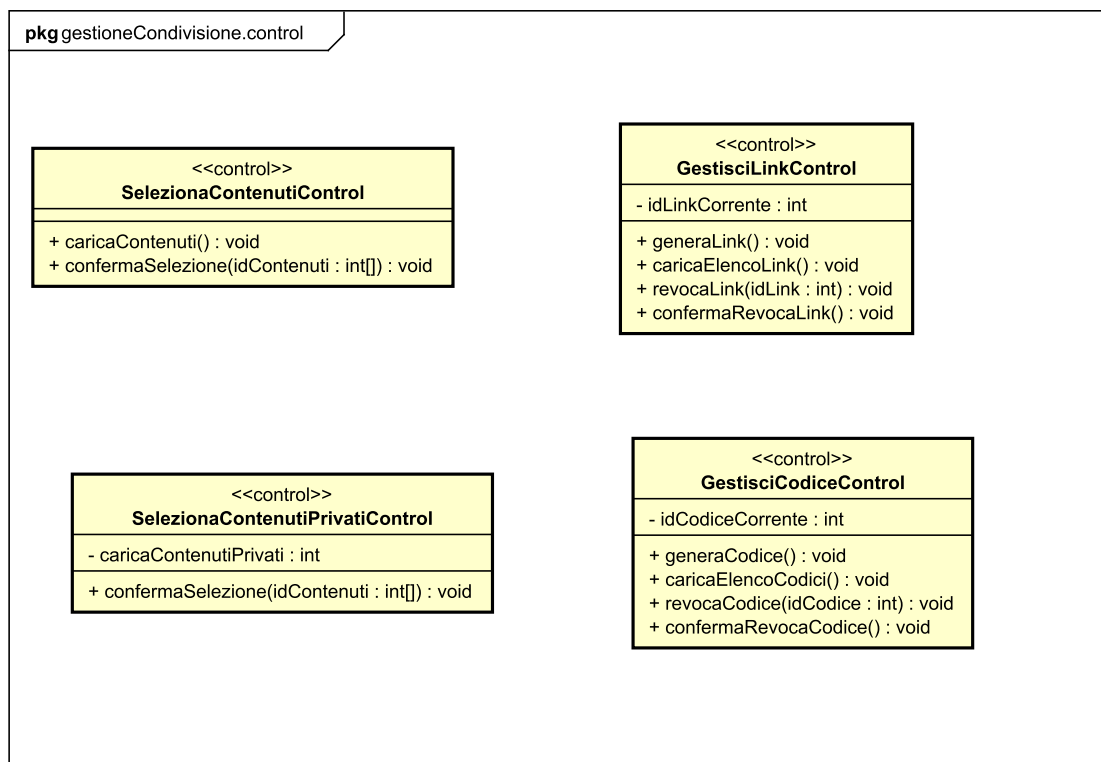


Figura 9: Diagramma UML del package GestioneCondivisione.control

3.9 Consultazione.interface

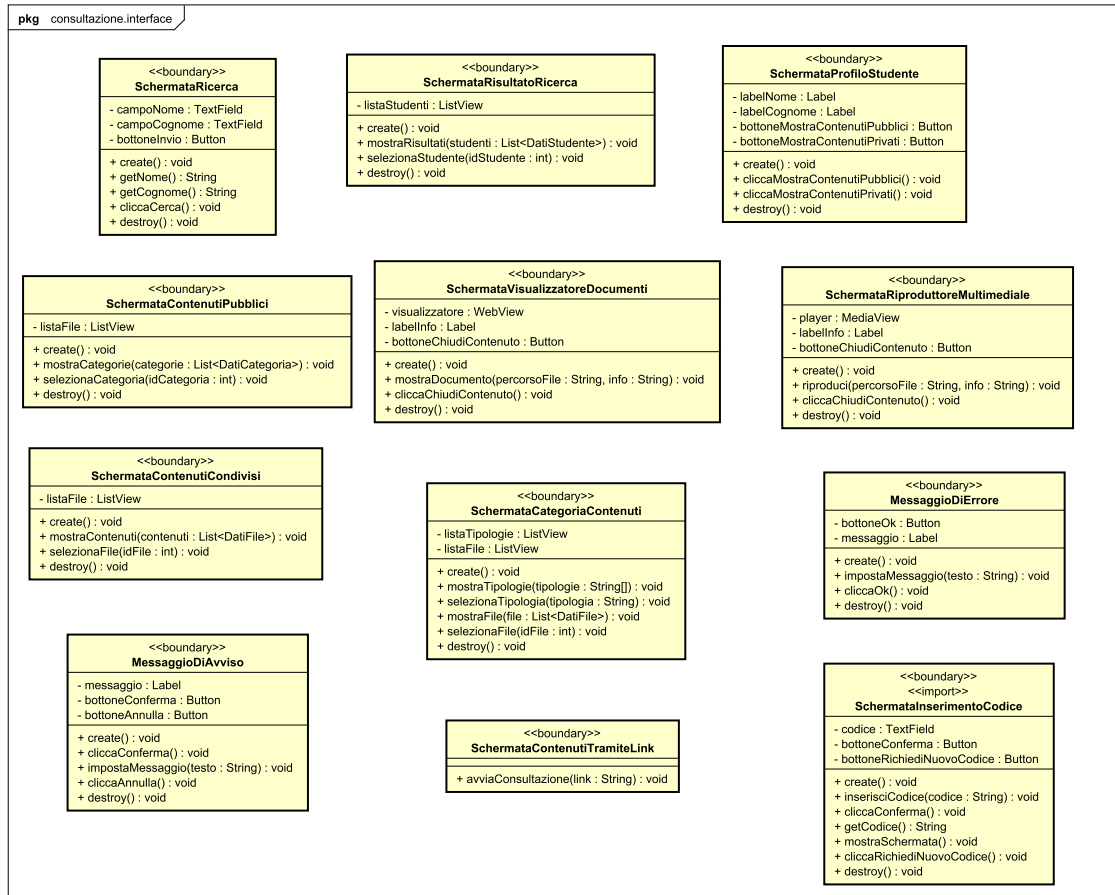


Figura 10: Diagramma UML del package Consultazione.interface

3.10 Consultazione.control

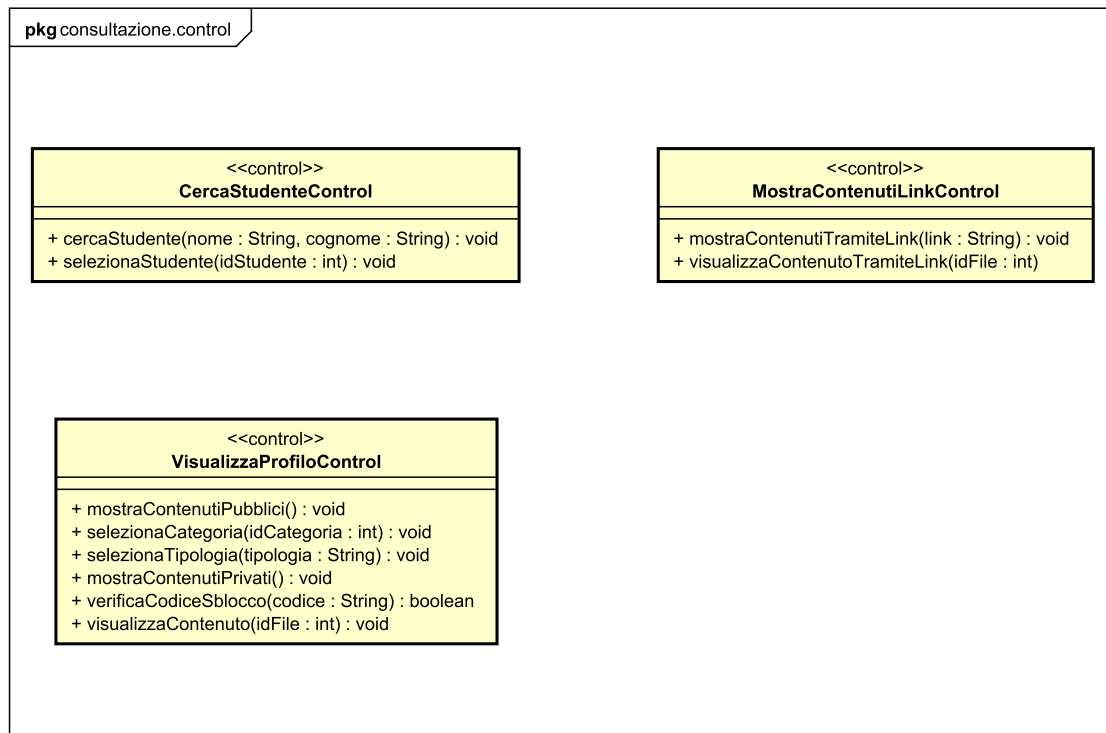


Figura 11: Diagramma UML del package Consultazione.control

3.11 GestioneOperazioniAutomatiche.interface

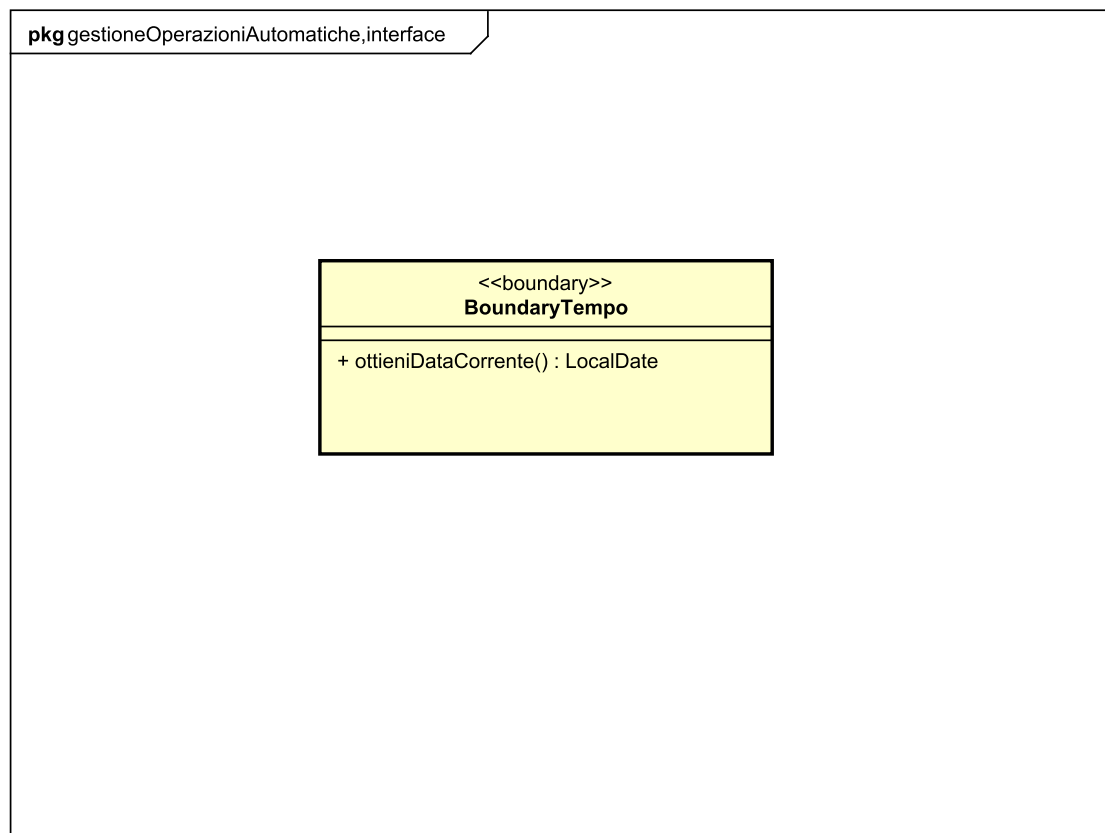


Figura 12: Diagramma UML del package GestioneOperazioniAutomatiche.interface

3.12 GestioneOperazioniAutomatiche.control

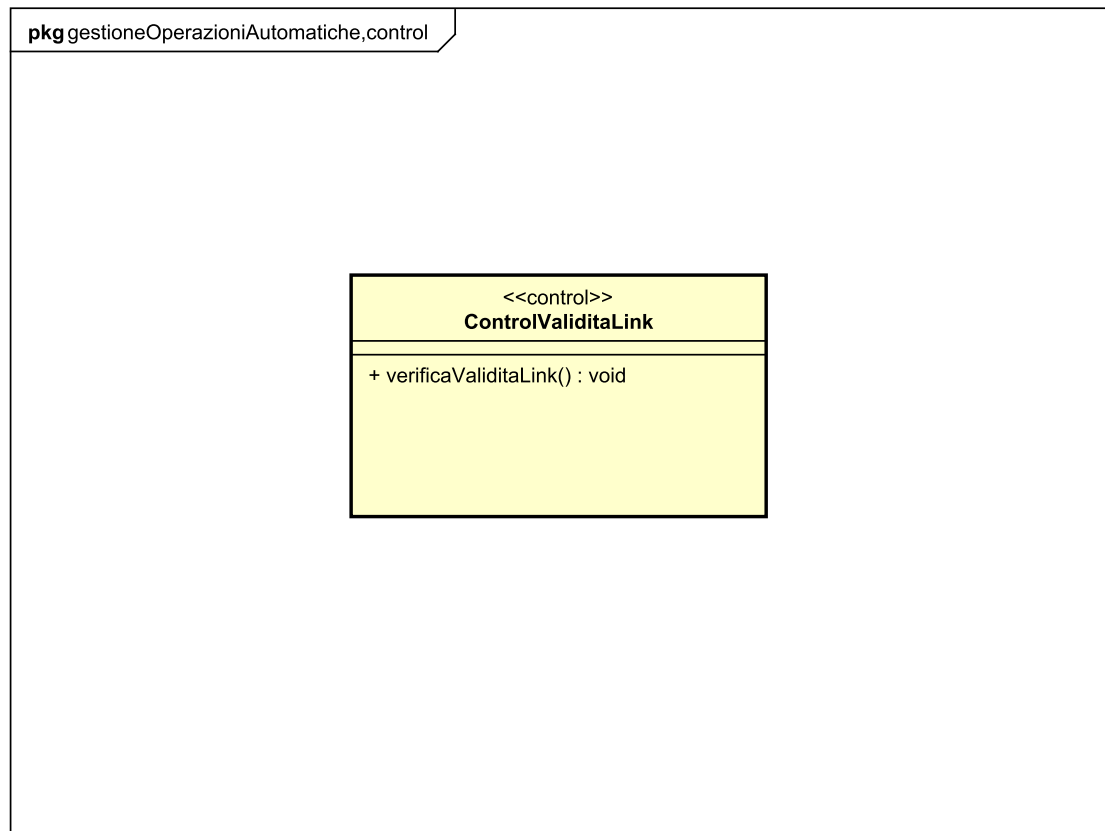


Figura 13: Diagramma UML del package GestioneOperazioniAutomatiche.control